SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Question 1

Concept Involved

Black-box test case design – Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), robustness testing and random testing, applied to the amount field of a fund-transfer function.

Definition

Black-box testing derives test cases from the specification (inputs and expected outputs) without looking at the code. **Equivalence Class Partitioning** divides the input domain into classes that the program treats in the same way, so one representative test per class is enough. **Boundary Value Analysis** selects values at and next to the edges of each class, because defects cluster at boundaries (for example, using < in place of ≤). **Random testing** picks inputs arbitrarily from the whole domain and does not use the specification to decide which values matter.

Scenario / Given Data

Item	Value (assumed for the analysis)
Function under test	Fund transfer – amount field (whole rupees)
Minimum transfer limit	₹100 (specification: 100 ≤ amount ≤ 50,000)
Maximum transfer limit	₹50,000 per transaction
Other conditions	Account must be active; balance ≥ amount; daily limit ₹1,00,000
Reported problem	Frequent failures at the minimum and maximum limit values
Current strategy	Random amounts plus a few typical values such as ₹5,000

The limits are assumed for demonstration; the same method applies to the limits in the bank's real specification.

Answer (Step-by-Step Solution)

Step 1: Critical analysis of the current test design

- **Random values rarely hit boundaries.** For amounts from 0 to 1,00,000 there are 1,00,001 possible values but only six boundary values (99, 100, 101, 49,999, 50,000, 50,001). One random test hits a boundary with probability 6 / 1,00,001 ≈ 0.006 %; about 11,500 random tests are needed for a 50 % chance of hitting one even once.
- **No systematic partitioning.** Invalid classes (zero, negative, non-numeric, empty, insufficient balance) were never designed, so the expected result of a test is often unknown.
- **Typical boundary defect.** A developer writes if (amount > 100 && amount < 50000) while the specification says 100 ≤ amount ≤ 50,000. The valid amounts 100 and 50,000 are then wrongly rejected. Only tests at exactly 100 and 50,000 expose this, and typical values such as ₹5,000 never do.
- **No traceability.** Test cases are not linked to requirements, so nobody can say which limit has been verified.

Step 2: Equivalence class partitioning

Class	Condition	Type	Representative value	Expected result
EC1	1 ≤ amount ≤ 99	Invalid	50	Rejected: “below minimum limit”
EC2	100 ≤ amount ≤ 50,000	Valid	5,000	Transfer successful
EC3	amount > 50,000	Invalid	60,000	Rejected: “exceeds maximum limit”
EC4	amount ≤ 0 (zero or negative)	Invalid	−500	Rejected: “invalid amount”
EC5	Non-numeric or empty input	Invalid	“abc”	Rejected: “enter a valid number”
EC6	Valid amount but balance < amount	Invalid	₹5,000 with balance ₹2,000	Rejected: “insufficient balance”

Step 3: Boundary value analysis

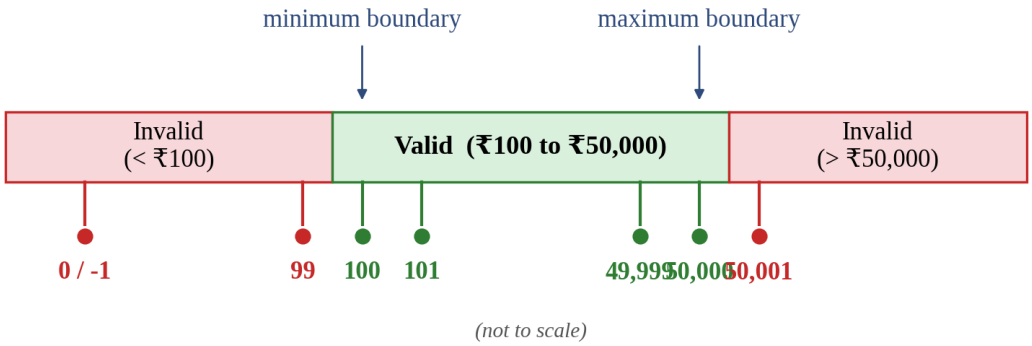


Fig. 1: Boundary values for the transfer amount (minimum ₹100, maximum ₹50,000)

Boundary	Test value (₹)	Type	Expected result
Just below minimum	99	Invalid	Rejected
Minimum	100	Valid	Accepted (fails if the code uses > 100)
Just above minimum	101	Valid	Accepted
Just below maximum	49,999	Valid	Accepted
Maximum	50,000	Valid	Accepted (fails if the code uses < 50,000)
Just above maximum	50,001	Invalid	Rejected
Robustness values	0 and −1	Invalid	Rejected
Balance boundary	Amount 5,000 with balance 5,000 and 4,999	Valid / Invalid	Accepted / Rejected (insufficient balance)

Step 4: Is random testing effective? Retain or replace?

Criterion	Random testing	ECP + BVA
Chance of hitting a boundary	≈ 0.006 % per test	Certain – boundaries are designed in
Tests needed for the amount field	Thousands, no guarantee	About 14
Expected result known in advance	Often not	Yes, taken from the specification
Repeatability and traceability	Low	High – each test maps to a requirement
Finds unexpected input behaviour	Sometimes	Limited to designed cases

Decision: random testing should be **replaced as the primary technique** and **retained only as a small supplement** (about 10 % of the effort), for example automated fuzz inputs with very large numbers, special characters and rapid repeated clicks, always with an automatic check (balance before and after the transfer) to decide pass or fail.

Step 5: Structured and optimised testing approach

1. List every limit and rule in the specification (minimum, maximum, daily limit, balance, account status).
2. Apply ECP to create valid and invalid classes for each input (Step 2).
3. Apply BVA at every boundary: min−1, min, min+1, max−1, max, max+1, plus 0 and −1 (Step 3).
4. Use a decision table for combinations of amount, account status and balance:

Condition / Action	R1	R2	R3	R4
Amount within 100 – 50,000?	Y	N	Y	Y
Account active?	Y	–	N	Y
Balance ≥ amount?	Y	–	–	N
Action	Transfer	Reject (limit)	Reject (inactive)	Reject (funds)

5. Add error guessing: double-click on “Transfer”, session time-out during transfer, amount with leading zeros (0100) or decimals (100.50).
6. Add a small automated random/fuzz run as a supplement.
7. Automate the ECP and BVA tests in the regression suite so that every release re-checks all limits.

Step 6: Justification – effectiveness and cost

Approach	Tests for the amount field	Boundary defects detected
Exhaustive testing	1,00,001	All, but impractical
Random testing (current)	e.g. 100 random tests	About 0.6 % chance of touching any boundary
ECP + BVA (proposed)	About 14	All off-by-one boundary defects, every class covered

The proposed set is more than 99.9 % smaller than exhaustive testing and, unlike random testing, is guaranteed to examine every boundary where the reported failures occur.

Application

The same approach is used for payment-gateway and UPI limits, ATM withdrawal limits, loan, insurance-premium and income-tax slab calculations, and any feature where a limit decides the result.

Conclusion

Random testing is unsuitable as the main strategy because it almost never reaches the exact limit values where the transfers fail. Equivalence class partitioning and boundary value analysis, supported by a decision table, error guessing and a small random supplement, give a structured, traceable and optimised test set of about 14 tests that maximises defect detection for the fund-transfer function.

Question 2

Concept Involved

White-box testing – control flow graph (CFG), statement, branch and path coverage, cyclomatic complexity, basis path testing and test adequacy criteria.

Definition

White-box testing derives test cases from the internal structure of the code. A **control flow graph** shows the module as nodes (statements or blocks) and edges (flow of control). **Test adequacy criteria** decide when testing is sufficient: statement coverage = executed statements / total statements; branch coverage = decision outcomes executed / total outcomes; path coverage = paths executed / total paths. **Cyclomatic complexity** $V(G) = E - N + 2$ = number of decisions + 1, and gives the number of independent paths.

Scenario / Given Data

The healthcare module is not given in detail, so a conceptual prescription-dose module is assumed. Dose is limited to 100 units and a patient with an allergy must not be dispensed the drug.

Node	Statement
N1	read(age, weight, allergic); dose = 0.3 × weight
N2 (D1)	if (age < 18)
N3	dose = 0.5 × weight
N4 (D2)	if (allergic)
N5	show_alert(); return
N6 (D3)	if (dose > 100)
N7	dose = 100
N8	dispense(dose)

Existing tests: T1: age 10, weight 250, allergic = Yes (alert). T2: age 30, weight 400, allergic = No (dose capped at 100, dispensed). Critical defects appeared in production on the normal dispensing path.

Answer (Step-by-Step Solution)

Step 1: Conceptual control flow graph

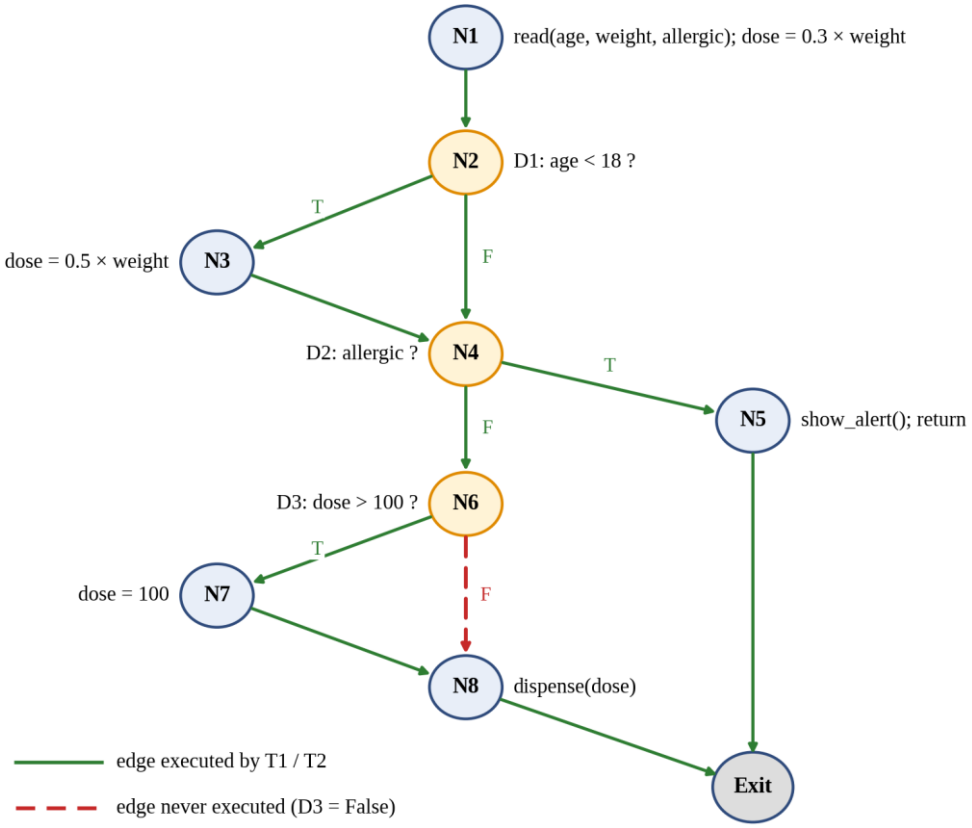


Fig. 2: CFG of the dose module – green: edges executed by T1/T2, red: edge never executed

Nodes N = 9 (including Exit), edges E = 11, so V(G) = E − N + 2 = 11 − 9 + 2 = **4** (three decisions + 1). At least 4 tests are needed to cover all independent paths.

Step 2: Coverage achieved by the existing tests

Criterion	Calculation	Result
Statement coverage	T1 runs N1–N5; T2 runs N1, N2, N4, N6, N7, N8 → all 8 nodes executed	8 / 8 = 100 %
Branch coverage	D1: T, F covered; D2: T, F covered; D3: T covered, F not covered	5 / 6 = 83.3 %
Path coverage	Total paths = 6; paths executed = 2 (T1 and T2)	2 / 6 = 33.3 %
Independent paths	V(G) = 4; paths executed = 2	2 / 4 = 50 %

Analysis: 100 % statement coverage looks complete but is misleading. Every statement ran at least once, yet no statement belongs only to the “dose ≤ 100” branch (an if without else), so the most common real-world case – a normal dose dispensed without capping – was never executed.

Step 3: Gaps – missing paths

Path	Decisions (D1, D2, D3)	Meaning	Tested?
P1	F, F, F	Adult, no allergy, normal dose → dispense	No (most common path)
P2	T, F, F	Child, no allergy, normal dose → dispense	No
P3	F, T	Adult with allergy → alert	No
P4	F, F, T	Adult, dose too high → capped to 100	Yes (T2)
P5	T, F, T	Child, dose too high → capped to 100	No
P6	T, T	Child with allergy → alert	Yes (T1)

Four of the six paths are untested, including the two normal-dispensing paths. This explains why defects escaped to production.

Step 4: Improved testing strategy

- Define adequacy criteria in advance: 100 % statement, 100 % branch, all 6 paths (feasible because the module has no loops) and boundary values of every condition.
- Use basis path testing: V(G) = 4 gives the minimum number of independent-path tests; extend to all 6 paths because the module is small and safety-critical.
- Add boundary tests for each condition (age 17/18, dose exactly 100 and just above 100).
- Run the suite with a coverage tool and review every uncovered line or branch.
- Prioritise by patient-safety risk: allergy alert and dose cap first.

TC	Age	Weight	Allergic	Path / purpose	Expected output
TC1	30	200	No	P1	Dose 60 dispensed
TC2	10	100	No	P2	Dose 50 dispensed
TC3	30	200	Yes	P3	Alert shown, nothing dispensed
TC4	30	400	No	P4	Dose capped, 100 dispensed
TC5	10	250	No	P5	Dose capped, 100 dispensed
TC6	10	100	Yes	P6	Alert shown, nothing dispensed
TC7	17	100	No	Boundary: age < 18	Child dose 50
TC8	18	100	No	Boundary: age < 18	Adult dose 30
TC9	10	200	No	Boundary: dose = 100	Dose 100 dispensed, cap not applied
TC10	10	201	No	Boundary: dose > 100	Dose 100.5 capped to 100

Criterion	Before (T1, T2)	After (TC1 – TC10)
Statement coverage	100 %	100 %
Branch coverage	83.3 %	100 %
Path coverage	33.3 %	100 %
Independent paths (V(G) = 4)	2 / 4	4 / 4

Step 5: How the approach improves reliability and reduces risk

- Every decision outcome and every path is executed, so the common dispensing path and the cap logic are verified before release.

- Allergy handling and dose limiting, the two patient-safety rules, are each tested on both the child and the adult route.
- Boundary tests catch off-by-one errors in conditions such as age < 18 or dose > 100.
- Coverage becomes measurable, so the team can show exit criteria were met; untested paths fall from 66.7 % to 0 %.
- Only 10 tests are needed, so the improved strategy is inexpensive and can be repeated as a regression suite.

Application

Path and branch coverage are used in medical-device software, drug-dosage calculators, hospital billing, banking and avionics, where an unexecuted path can cause serious harm.

Conclusion

The earlier tests achieved 100 % statement coverage but only 83.3 % branch and 33.3 % path coverage, so the normal dispensing paths were never tested. By building the control flow graph (V(G) = 4), enumerating the six paths and adding boundary tests, the improved suite of 10 test cases reaches complete statement, branch and path coverage, which makes the module more reliable and lowers the risk of production failures.

Question 3

Concept Involved

Black-box testing of a multi-input system – Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), decision table testing and test completeness.

Definition

When the output depends on several inputs together, testing each field alone is not enough. **ECP** groups the values of each field into valid and invalid classes; **BVA** tests the edges of each class; **decision table testing** lists every combination of conditions with the action the system must take, so that interactions between fields are tested. Test completeness means every class, boundary and rule of the specification is covered by at least one test case.

Scenario / Given Data

Field	Valid range	Rule (assumed for the analysis)
Price (₹)	1 – 10,000	Price per unit
Quantity	1 – 100	Whole number
Discount (%)	0 – 50	Applied only when quantity ≥ 5 (bulk order)
Order total	Price × quantity × (1 − discount / 100)	Must never be zero or negative

Nominal test values: price 500, quantity 10, discount 10 % → total = 500 × 10 × 0.90 = ₹4,500. The testers checked one field at a time and missed errors that occur only for particular combinations.

Answer (Step-by-Step Solution)

Step 1: Limitations of the current black-box approach

- **Single-field testing:** each field was varied while the others stayed fixed, so interactions (for example a discount entered with quantity below 5) were never tried.

- **Only typical values:** boundary values such as price 10,000 or discount 50 % were not selected, and invalid classes were weak.
- **Exhaustive testing is impossible:** 10,000 prices × 100 quantities × 51 discounts = 5.1 crore combinations.
- **No written combination rules:** without a decision table nobody knows which combinations exist, so missing ones go unnoticed.

Step 2: Equivalence class partitioning

Field	Valid class	Invalid classes
Price	1 – 10,000	Below 1 (0, negative); above 10,000; non-numeric
Quantity	1 – 100	Below 1; above 100; decimal (2.5); non-numeric
Discount	0 – 50	Below 0; above 50; non-numeric

With three classes per field (below, valid, above) strong ECP gives 3 × 3 × 3 = 27 combinations, of which only one is fully valid. Weak ECP uses 1 valid test plus one test per invalid class (single-fault assumption), which is cheaper and is combined with BVA below.

Step 3: Robust boundary value analysis (6n + 1 = 19 tests)

For each field the values min−1, min, min+1, max−1, max, max+1 are tested while the other two fields hold their nominal values (price 500, quantity 10, discount 10):

ID	Price	Quantity	Discount %	Expected result
T01	500	10	10	Total 4,500.00 (nominal)
T02	0	10	10	Rejected – invalid price
T03	1	10	10	Total 9.00
T04	2	10	10	Total 18.00
T05	9,999	10	10	Total 89,991.00
T06	10,000	10	10	Total 90,000.00
T07	10,001	10	10	Rejected – invalid price
T08	500	0	10	Rejected – invalid quantity
T09	500	1	10	Total 500.00 (no discount, quantity < 5)
T10	500	2	10	Total 1,000.00 (no discount)
T11	500	99	10	Total 44,550.00
T12	500	100	10	Total 45,000.00
T13	500	101	10	Rejected – invalid quantity
T14	500	10	−1	Rejected – invalid discount
T15	500	10	0	Total 5,000.00
T16	500	10	1	Total 4,950.00
T17	500	10	49	Total 2,550.00
T18	500	10	50	Total 2,500.00
T19	500	10	51	Rejected – invalid discount

Step 4: Decision table testing for combinations

Four conditions are used: C1 price valid, C2 quantity valid, C3 discount valid, C4 quantity ≥ 5. Sixteen combinations collapse into five rules because an invalid field makes the other conditions irrelevant ("–").

Condition / Action	R1	R2	R3	R4	R5
C1: Price valid?	N	Y	Y	Y	Y
C2: Quantity valid?	–	N	Y	Y	Y
C3: Discount valid?	–	–	N	Y	Y
C4: Quantity ≥ 5?	–	–	–	Y	N
A1: Total with discount				X	
A2: Total without discount					X
A3: Reject with error	X	X	X		
Combinations covered	8	4	2	1	1
Test cases	T02, T07	T08, T13	T14, T19	T01, T21	T09, T10, T20

Rule R5 is exactly the type of combination that was missed (a discount entered with a small quantity). Two more tests check the bulk threshold, which is a boundary of condition C4, and three check data types:

ID	Price	Quantity	Discount %	Expected result
T20	500	4	10	Total 2,000.00 (no discount)
T21	500	5	10	Total 2,250.00 (discount applied)
T22	“abc”	10	10	Rejected – invalid data type
T23	500	2.5	10	Rejected – quantity must be whole
T24	500	10	“abc”	Rejected – invalid data type

Step 5: Revised test design strategy

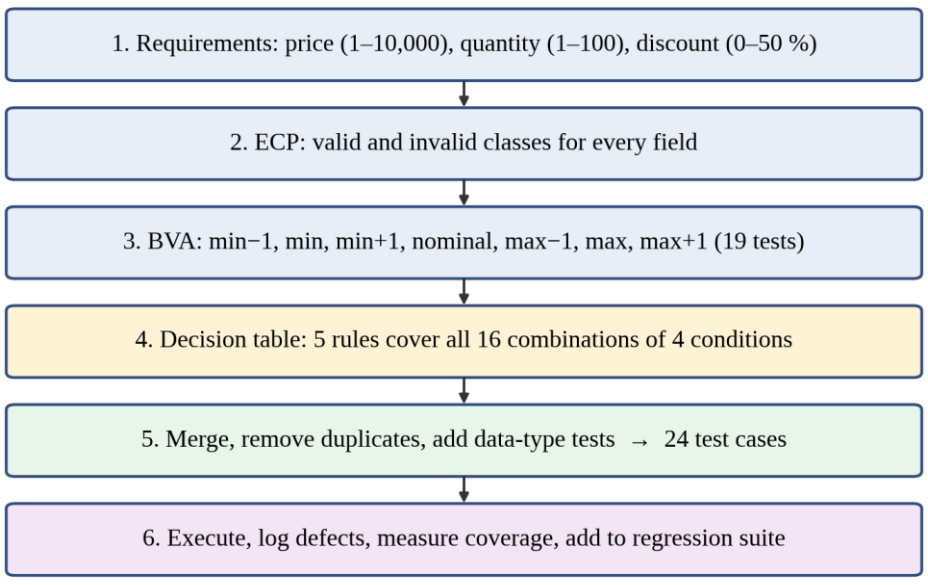


Fig. 3: Structured test design strategy for price, quantity and discount

Technique	Tests	Purpose
Robust BVA (6n + 1)	19	Boundary of every field (also covers rules R1 – R5)
Decision table extras	2	Bulk-order threshold (quantity 4 and 5)
Data-type tests	3	Non-numeric and decimal input
Total	**24**	All classes, boundaries and rules

Step 6: Justification

- **Defect detection:** boundary errors (≤ written as <), wrong discount eligibility and invalid-input handling are each targeted by a specific test.
- **Completeness:** all 16 condition combinations (5 rules), all 18 boundary points and all invalid classes are covered, with every test traceable to a rule.
- **Efficiency:** 24 tests replace 5.1 crore combinations, a reduction of more than 99.9999 %, so the suite can be run in every build.

Application

The same combination of ECP, BVA and decision tables is used for tax and GST calculators, insurance premium rules, shipping-cost and discount engines, loan eligibility and any form with dependent fields.

Conclusion

Testing one field at a time cannot find defects that appear only for particular combinations. Equivalence class partitioning and robust boundary value analysis cover each field, and decision table testing covers the combinations of conditions. The resulting 24 test cases give complete, traceable coverage of the price, quantity and discount logic at a tiny fraction of the cost of exhaustive testing.

Question 4

Concept Involved

White-box testing – control flow graph (CFG) construction, independent paths, basis path testing, loop testing, coverage criteria and cyclomatic complexity.

Definition

An **independent path** is a path through the CFG that introduces at least one new edge. **Basis path testing** designs one test for each independent path; their number equals the cyclomatic complexity V(G) = E − N + 2 = number of decisions + 1. **Loop testing** checks a loop at skip, one pass, two passes, a typical number of passes and the maximum−1, maximum and maximum+1 passes, because the number of paths through a loop is otherwise unbounded.

Scenario / Given Data

Node	Statement
N1	total = 0; i = 1
N2 (D1)	while (i <= n)
N3 (D2)	if (x[i] > 0)
N4 (D3)	if (x[i] > 100)
N5	total = total + 100
N6	total = total + x[i]
N7	i = i + 1
N8	return total

A module adds the positive values of an array, capping each value at 100. It has a loop with nested conditions (assumed for the analysis):

Team's tests: TA: n = 2, x = [50, −5] (result 50). TB: n = 1, x = [10] (result 10). The team reports only partial path coverage.

Answer (Step-by-Step Solution)

Step 1: Control flow graph and cyclomatic complexity

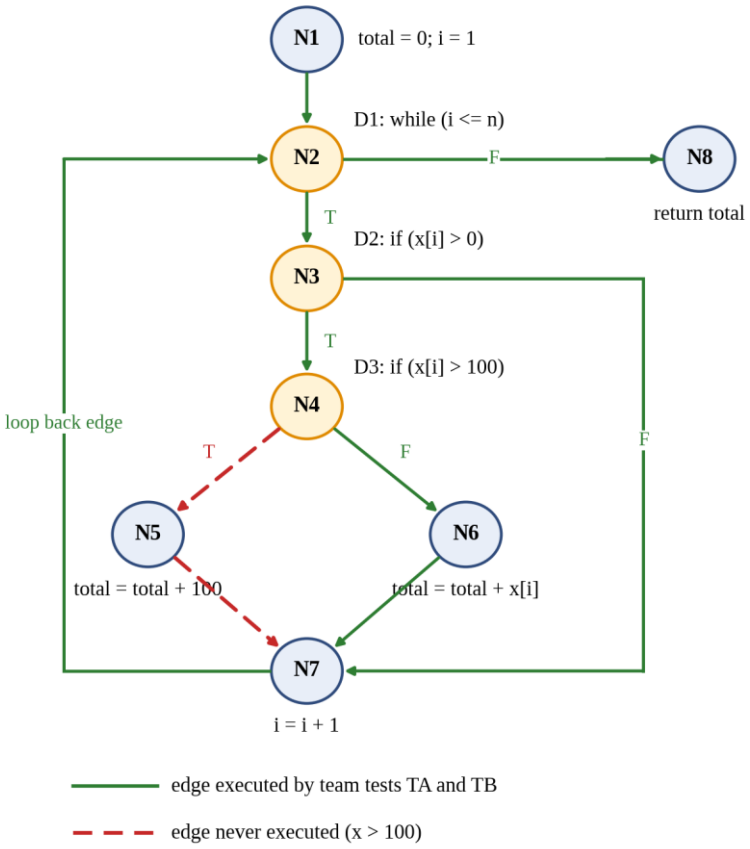


Fig. 4: CFG of the loop module – red edges were never executed by the team's tests

Nodes N = 8, edges E = 10 (including the loop back edge N7 → N2), so V(G) = 10 − 8 + 2 = **4**; the same value is obtained from three decisions + 1. Four independent paths must be tested.

Step 2: Evaluation of the current white-box approach

Criterion	Observation	Result
Statement coverage	N5 (total = total + 100) never executed	7 / 8 = 87.5 %
Branch coverage	D3 true outcome (x > 100) never taken	5 / 6 = 83.3 %
Independent paths	Only P2 and P3 exercised	2 / 4 = 50 %
Loop testing	Only 1 and 2 iterations; skip and boundary loops not tested	2 of 7 cases

Step 3: Gaps in CFG construction and path testing

- The graph was drawn without the **loop back edge** or with the nested if treated as one decision, so V(G) can easily be underestimated (for example 2 instead of 4).
- **Independent paths were not enumerated**, so test data were chosen at random instead of to drive each condition true and false.
- **Loop boundaries were ignored:** the case n = 0 (loop skipped) and large values of n were never run.
- **Impact on defect detection:** the cap logic could be wrong (for example total = 100 instead of total + 100) without any test failing; an empty input or an off-by-one in the loop bound would go unnoticed; high statement or edge coverage figures would still look acceptable.

Step 4: Independent paths (basis set) and test cases

Path	Node sequence	Condition outcomes	Test data	Expected
P1	N1–N2–N8	Loop not entered	n = 0	0
P2	N1–N2–N3–N7–N2–N8	$x \leq 0$	n = 1, x = [−5]	0
P3	N1–N2–N3–N4–N6–N7–N2–N8	$0 < x \leq 100$	n = 1, x = [50]	50
P4	N1–N2–N3–N4–N5–N7–N2–N8	$x > 100$	n = 1, x = [150]	100

Step 5: Loop testing and condition boundaries

ID	Test data	Purpose	Expected
L1	n = 2, x = [50, 150]	Two passes	150
L2	n = 5, x = [10, 20, −1, 200, 30]	Typical number of passes, all inner paths	160
L3	n = 99, every x = 10	Maximum − 1 passes (maximum n = 100)	990
L4	n = 100, every x = 10	Maximum passes	1,000
L5	n = 101	Maximum + 1 (must be rejected or handled)	Error / handled
C1	n = 1, x = [0]	Boundary x = 0 (not added)	0
C2	n = 1, x = [1]	Boundary x = 1	1
C3	n = 1, x = [100]	Boundary x = 100 (D3 false)	100
C4	n = 1, x = [101]	Boundary x = 101 (D3 true, capped)	100
M1	n = 3, x = [50, −5, 150]	Different inner paths in successive iterations	150

Together with P1 – P4 the strategy uses 14 test cases.

Step 6: Justification using coverage criteria and cyclomatic complexity

Criterion	Before (TA, TB)	After (14 tests)
Statement coverage	87.5 %	100 %
Branch coverage	83.3 %	100 %
Independent paths (V(G) = 4)	2 / 4 = 50 %	4 / 4 = 100 %
Loop cases (0, 1, 2, typical, max−1, max, max+1)	2 / 7	7 / 7

- V(G) = 4 is the **minimum** number of tests for complete independent-path coverage; the loop and boundary tests add the cases that V(G) alone does not capture.
- V(G) = 4 is below the usual limit of 10, so the module is simple enough to be tested completely and fully understood; a value above 10 would call for refactoring.
- **Expected outcome:** all four paths, both outcomes of every decision and all loop situations are executed, so cap errors, empty input, wrong loop bound and condition boundary errors are detected.

Application

Path and loop testing are applied to data-migration tools, billing and report generators, search and sorting routines, embedded controllers and any code with nested loops and conditions.

Conclusion

The team's two tests exercised only half of the independent paths and left the capping branch and the loop boundaries untested. By drawing the complete CFG (V(G) = 4), deriving the four basis paths and adding loop and boundary tests, a set of 14 test cases gives full statement, branch and independent-path coverage and a much higher chance of finding hidden defects.

Question 5

Concept Involved

Hybrid test design – combining black-box techniques (ECP, BVA) with white-box techniques (CFG, path testing, condition coverage); test adequacy criteria; coverage metrics; defect prevention.

Definition

A **hybrid strategy** designs tests from the specification (black box) and then uses code coverage (white box) to find what those tests missed, so each technique covers the weakness of the other. **Test adequacy criteria** are rules that decide when testing is sufficient, and **coverage metrics** measure how much of the specification (classes, boundaries) and the code (statements, branches, conditions, paths) has been exercised.

Scenario / Given Data

A loan-eligibility module of a financial application (assumed for the analysis). Specification: age 21 – 60 inclusive; monthly income ≥ ₹15,000; loan amount ≤ 10 × monthly income. The code also contains a rule that is not in the specification: amounts above ₹50,00,000 are flagged for audit.

Node	Statement
N1	read(age, income, amount)
N2 (D1)	if (age < 21 || age >= 60) ← defect: specification allows age 60
N3	reject("age")
N4 (D2)	else if (income < 15000)
N5	reject("income")
N6 (D3)	else if (amount > 10 × income)
N7	reject("amount")
N8 (D4)	if (amount > 5000000)
N9	flag_for_audit()
N10	approve()

Existing tests: E1: age 30, income 50,000, amount 1,00,000 (approve). E2: age 15 (reject). E3: age 30, income 10,000 (reject). E4: age 30, income 20,000, amount 5,00,000 (reject). Defects in production: 60-year-old applicants wrongly rejected, and failures for very large loans.

Answer (Step-by-Step Solution)

Step 1: Critical analysis of the integration of strategies

Observation	Problem
Black-box tests used typical values	Boundaries of age (60), income and amount were not selected; ECP and BVA were incomplete
White-box tests used arbitrary data	Paths were traversed with values such as 30 and 15, so the off-by-one at age 60 stayed hidden
Coverage was not compared with the test suite	The audit branch was never executed and nobody noticed
No adequacy or exit criteria	The number of test cases, not coverage, was used to decide that testing was complete
Techniques applied separately	No traceability between requirement, test case and code, and no feedback between the two sets

Step 2: Evaluation of adequacy criteria and coverage metrics (existing tests E1 – E4)

Criterion	Result	Gap
Statement coverage	9 / 10 = 90 %	N9 (flag_for_audit) never executed
Branch (decision) coverage	7 / 8 = 87.5 %	D4 true outcome never taken
Condition coverage (10 condition outcomes)	8 / 10 = 80 %	age ≥ 60 and amount > 50,00,000 never true, so the boundary defect is hidden
Path coverage (V(G) = 5)	4 / 5 = 80 %	Approve-with-audit path missing
Boundary values selected	0 / 11	No boundary of age, income or amount tested

The metrics were not properly applied: decision D1 shows both outcomes covered (age 15 and age 30), yet the second condition of the compound decision was never true.

Step 3: Improved hybrid testing strategy

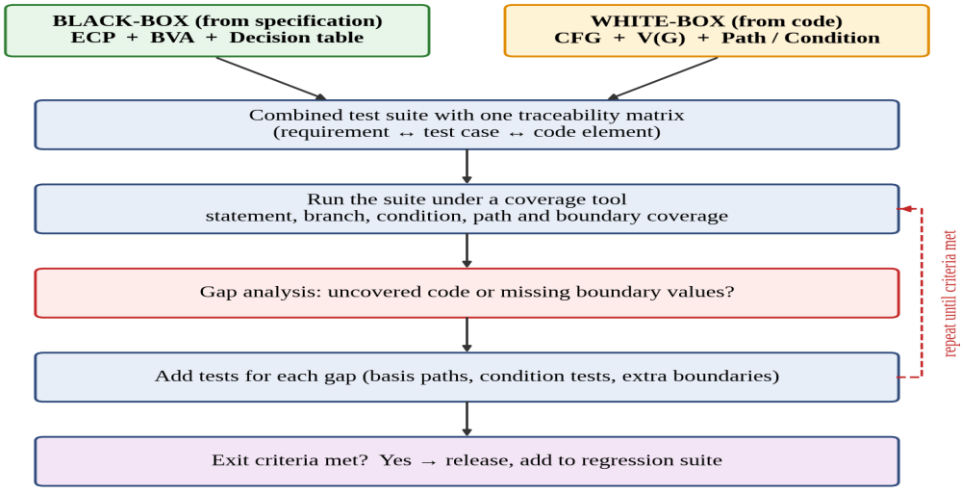


Fig. 5: Hybrid black-box and white-box testing strategy

1. **Black box first:** apply ECP and BVA to every input of the specification (age, income, amount) and build a decision table for the combinations.
2. **Build the CFG** of the code and compute V(G) = 4 decisions + 1 = 5.
3. **Execute the black-box suite under a coverage tool** and record statement, branch, condition and path coverage.
4. **Gap analysis:** every uncovered statement, branch, condition or path gets a new white-box test, chosen at boundary values wherever a condition is involved.
5. **Exit criteria:** 100 % statement, branch and condition coverage, all basis paths executed, all boundary values tested, and every requirement mapped to at least one test.

Step 4: Hybrid test suite

ID	Age	Income (₹)	Amount (₹)	Expected result	Technique
H1	20	30,000	1,00,000	Reject (age)	BVA
H2	21	30,000	1,00,000	Approve	BVA
H3	22	30,000	1,00,000	Approve	BVA
H4	59	30,000	1,00,000	Approve	BVA
H5	60	30,000	1,00,000	Approve (defect found)	BVA
H6	61	30,000	1,00,000	Reject (age)	BVA
H7	30	14,999	10,000	Reject (income)	BVA
H8	30	15,000	10,000	Approve	BVA
H9	30	15,001	10,000	Approve	BVA
H10	30	20,000	2,00,000	Approve (= 10 × income)	BVA
H11	30	20,000	2,00,001	Reject (amount)	BVA
H12	30	7,00,000	60,00,000	Approve and audit flag	White box (D4)

Step 5: Coverage after the hybrid strategy

Criterion	Before (E1 – E4)	After (H1 – H12)
Statement coverage	90 %	100 %
Branch coverage	87.5 %	100 %
Condition coverage	80 %	100 %
Path coverage	4 / 5	5 / 5
Boundary values tested	0 / 11	11 / 11

Step 6: Justification – better coverage and defect prevention

Defect	Found by	Reason
Age 60 wrongly rejected (>= 60)	BVA (black box)	The boundary comes from the specification; arbitrary white-box data misses it
Failure on the audit branch	Branch / path coverage (white box)	The rule is not in the specification, so black-box tests have no reason to reach it
Weak compound condition (||)	Condition coverage	Each condition must be true and false independently

- **Better coverage:** a traceability matrix links requirement, test and code element, and the coverage tool shows what is still untested.
- **Defect prevention:** requirement and code reviews with a checklist for boundary conditions (< versus ≤), static analysis for off-by-one errors, root-cause analysis of every production defect, and a regression suite run on each build.
- **Objective exit criteria** replace the count of test cases, so testing stops only when the criteria are met.

Application

Hybrid testing is used in banking and loan systems, payment processing, insurance, trading platforms and other financial applications where both specification errors and hidden code paths cause costly defects.

Conclusion

Defects continued to appear because black-box and white-box tests were designed independently, boundary values and condition coverage were missed, and no adequacy criteria were enforced. A hybrid strategy that starts with ECP and BVA, uses the CFG and coverage reports to find gaps, and closes them with path and condition tests reaches 100 % statement, branch, condition and path coverage with only 12 test cases, and with reviews and regression testing it prevents the same defects from returning.